

nano-dates

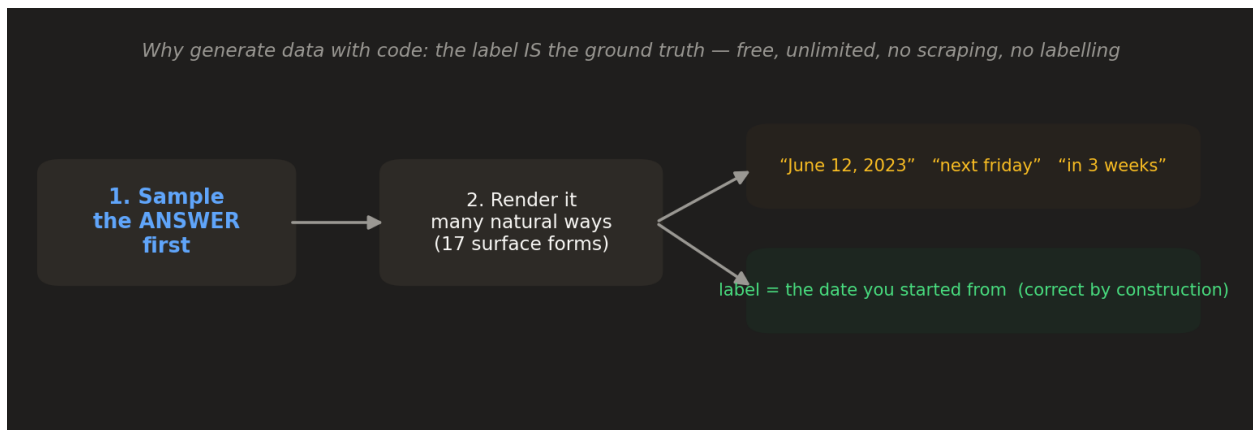
A 1M-parameter date parser trained entirely on data generated by code

nano-dates is a 1,016,960-parameter byte-level transformer that converts a natural date phrase into an ISO-8601 date (e.g. "the 3rd of July 2025" → 2025-07-03). It runs on a CPU in milliseconds, trains from scratch in about 30 seconds on one GPU, and every training example was produced by code — no scraping, no labelling, no distillation from a larger model.

Model: huggingface.co/vukrosic/nano-dates · Code: github.com/vukrosic/nano-dates

1 Why generate the data with code

A 1M-parameter model can't be a general assistant, but it can completely nail a task that is narrow and formally specified. Date→ISO is exactly that — and that makes the data free. You sample the answer first, then render it in many natural surface forms. Because you started from the answer, the label is correct by construction: strictly better than asking a big model to generate data (no verification, no cost, unlimited, and the label *is* the ground truth).

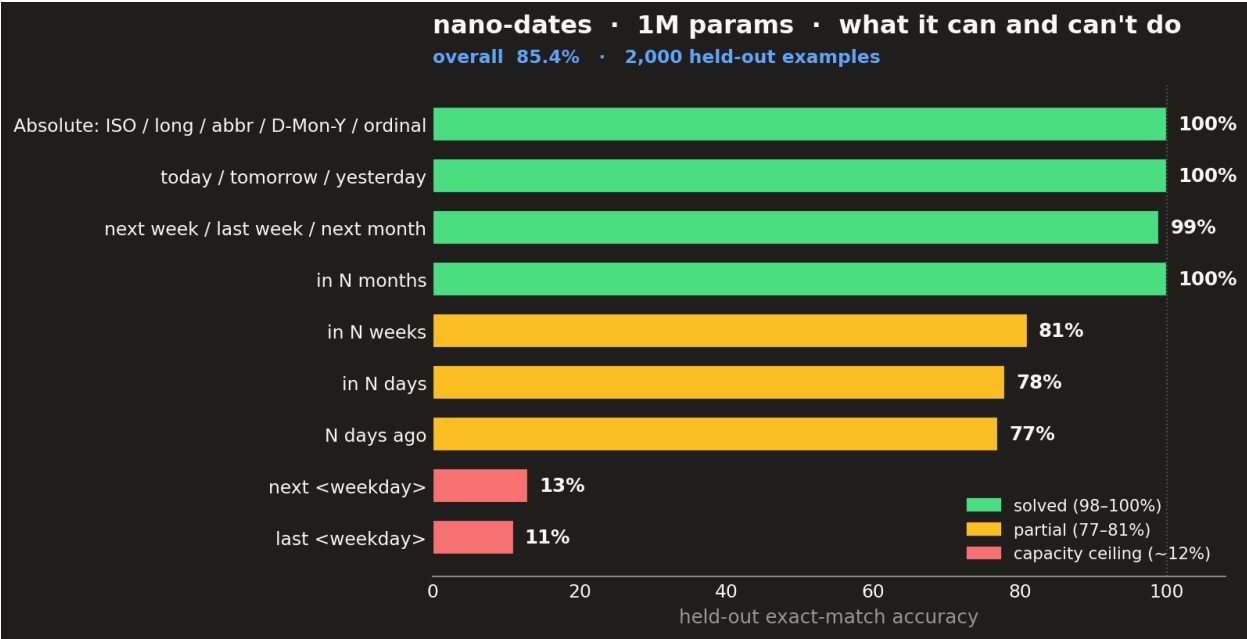


The model is given a reference date (today) at the start of the prompt, so relative phrases are computable from the input alone. Prompt format, byte-for-byte:

```
<reference ISO date> | <phrase> => <answer ISO date>
```

2 What it can and can't do

Held-out exact-match accuracy on 2,000 unseen examples (greedy decode). Overall: **85.4%**.



Capability	Examples	Accuracy
Parse absolute dates	June 12, 2023 · the 12th of June 2023	100%
Resolve simple relatives	today · tomorrow · next/last week · next month · in N months	98–100%
Variable-N day/week math	in N days · N days ago · in N weeks	77–81%
Weekday resolution	next friday · last tuesday	~12%

The clean limitation is **weekday resolution**. Resolving “next friday” means mapping an arbitrary date to its day-of-week and then doing modular-7 arithmetic — the most globally entangled computation in the set — and a 1M model doesn't have the capacity. Adding training didn't move it: it's a capacity ceiling, not undertraining.

3 The bug worth knowing about

An earlier version scored 100% on every absolute form — and it was cheating. The generator had set the prompt's reference date *equal to* the absolute answer, so the model learned to copy the ISO prefix instead of reading the phrase. The held-out eval confirmed the bug because it shared it. The only thing that caught it was feeding the model an input the generator could never produce: a mismatched reference date, on which the “perfect” model echoed the wrong date.

The fix: absolute forms now sample an answer date independent of today, so the only way to be right is to actually parse the phrase. This also lifted variable-N arithmetic from 32–45% to 77–81% — forcing real computation everywhere helped.

“Generated data being ‘correct by construction’ does not mean it isn't leaking the answer through a side channel. Your evaluation will happily confirm a model that learned your bug.”

4 Model & training

Parameters	1,016,960
Architecture	decoder-only transformer, pre-norm (RMSNorm, RoPE, GQA, SwiGLU)
Tokenizer	raw UTF-8 bytes (vocab 256, no vocab file)

dim / layers / heads	128 / 4 / 4 (2 KV heads), context 64 bytes
Training	prompt-masked SFT, 12k steps, batch 64, AdamW, cosine LR 3e-3
Data	100k code-generated pairs, 17 surface renderers
Final val loss	0.036

This is a method demonstration and a capability study, not a production date library — for real software, dateutil/chrono are exact and free. The value is showing how far a nano model gets on a formal task with synthetic data, and drawing a legible line at where its reasoning breaks. Vuk Rosi ■ · 2026 · MIT.